

Graphwise Stack Builder — Deployment & POC Guide

INTERNAL USE ONLY | v2.0.0 | Author: Kent Stroker

This is the single, cohesive runbook for standing up a Graphwise demo stack and configuring a customer POC on top of it. It replaces the overlapping guidance that used to be split across `DEPLOYMENT_GUIDE.md`, `QUICK-START.md`, and `STACK-AND-POC-GUIDE.md` (those files remain in place for reference).

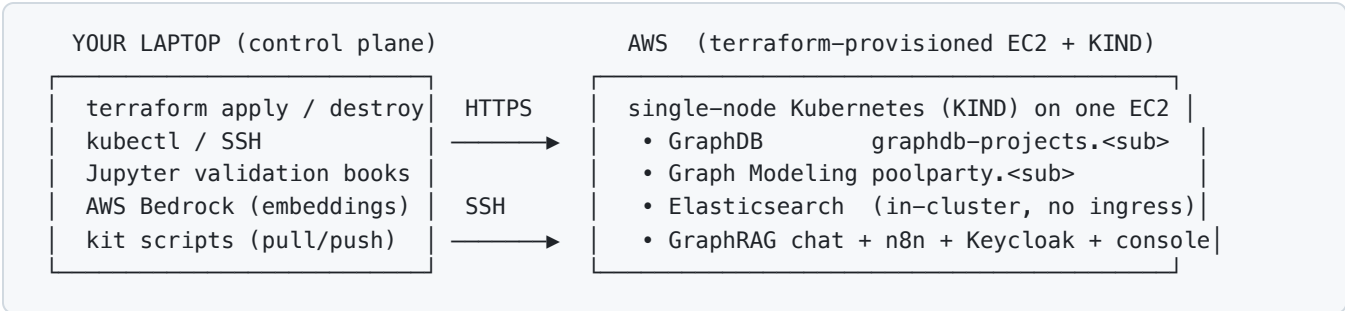
It is organized into two chapters:

- **Chapter 1 — Deployment** covers the **Graphwise Stack Builder (GSB)** only: provisioning the AWS stack with Terraform (EC2 + KIND + Helm) and operating it day-to-day. Read this once, then return to it for each new stack or rebuild.
- **Chapter 2 — POC Configuration** covers everything *after* the stack is up: loading data, wiring the n8n workflow engine, running the ingest pipelines, validating the knowledge graph, and activating the GraphRAG chatbot. Return to this chapter for each customer engagement.

This guide ships inside the downloadable **laptop kit** (`graphwise-stack-builder-kit-v2.0.0.tar.gz`) — **get it from Kent**, extract it, and run everything from within the extracted folder. The kit contains credentials and license keys valid for **internal Graphwise use only**. For a customer-facing stack, use proper 30-day licenses — **not** the internal kit licenses.

What you are building, and where

The knowledge graph and its services do **not** run on your laptop. GraphDB, Graph Modeling (Thesaurus + Extractor), Elasticsearch, Keycloak, n8n, and the pre-baked GraphRAG chat suite all run on a single-node Kubernetes (KIND) cluster on one AWS EC2 instance, provisioned and managed by Terraform.



Why a hosted stack. Graph Modeling, GraphDB, and the Graphwise GraphRAG suite are a multi-service Helm deployment with SSO, TLS, and inter-service wiring — far more than a laptop should run. Terraform makes it one repeatable command instead of hand-built infrastructure.

What **terraform apply** **stands up** (high level):

1. An EC2 host (Amazon Linux 2023, ARM64/Graviton) with Docker + **KIND** (Kubernetes-in-Docker) + Helm bootstrapped via cloud-init — a single-node cluster.
2. Two Helm installs deploy the **Graphwise umbrella** (Graph Modeling, two GraphDB instances, Elasticsearch, Keycloak, Semantic Workbench) and the **GraphRAG** suite (chatbot + conversation API + seeded n8n workflows), each on its own public subdomain with a Let's Encrypt cert and Keycloak SSO.
3. A pre-allocated Elastic IP + DNS wildcard give every service a stable URL under `<sub>.gw-pse.com`.

Base domain: `gw-pse.com` (Route 53 hosted zone in the deployment AWS account). Substitute your own values for `<sub>` (subdomain), `<eip>` (Elastic IP), and `<key.pem>` (your EC2 key) throughout.

Assumes the Graphwise PSE AWS account is already set up by your account admin. You do **not** create AWS accounts, IAM users, or the Route 53 zone — they already exist; you just consume them. What you need:

- The CSV files containing your AWS access keys (ID + secret).
- An IAM account that already has the permissions the full stack requires, including Bedrock and the IAM permissions Terraform needs.
- An **EC2 key pair you create** (Chapter 1, Deploy step 0) — Terraform attaches it to the instance for SSH; keep its `.pem` on your laptop (`$HOME/.ssh`).
- The internal Graphwise licenses, which ship in the kit: the **maven** user/pass, the **n8n Enterprise** key, and the license files `poolparty.key` , `graphdb.license` , `uv-license.key` .

A note on script paths. The laptop-side kit scripts live in your per-stack Terraform folder and are invoked as `./prep-scripts/<name>.sh` throughout this guide. (Older copies of the kit placed them under `scripts/` — if `./prep-scripts/` is not found, try `./scripts/`.) The stack-build scripts run on the EC2 from `~/gsb/prep-scripts/`.

Table of Contents

Chapter 1 — Deployment (Graphwise Stack Builder)

- [Kit Layout](#)
- [Kit Scripts](#)
- [Laptop Prerequisites \(one-time\)](#)
- [ZSH Profile SSH Convenience](#)
- [Deploy a New Stack](#)
- [After the Build: Pull Config](#)
- [Rebuild / Destroy-and-Rebuild Flow](#)
- [Auto-Shutdown and Restart](#)
- [Loading Staging Data](#)

- [Stack Tear Down](#)
- [Deployment Troubleshooting](#)
- [Kubernetes Dashboard and Grafana](#)

Chapter 2 — POC Configuration

- [POC Overview](#)
 - [Step 1 — Load GraphDB](#)
 - [Step 2 — Load Graph Modeling](#)
 - [Step 3 — Configure n8n](#)
 - [Step 4 — Structured Ingest](#)
 - [Step 5 — Unstructured Ingest](#)
 - [Step 6 — Annotate \(Extractor\)](#)
 - [Cross-workflow Consistency Requirements](#)
 - [Steps 7–9 — Validate](#)
 - [Step 10 — Activate the Chatbot](#)
 - [Step 11 — Prune n8n Execution History](#)
 - [Re-run Reference](#)
 - [POC Troubleshooting](#)
-
-

Chapter 1 — Deployment

Once three files are edited with your own credentials and stack-specific details, a single command (`terraform apply`) builds the entire stack, and a single command (`terraform destroy`) deletes it. The heavy lifting — charts, cluster scripts — is cloned onto the EC2 automatically by Terraform cloud-init; you never copy those by hand.

The only AWS resources you provision in the Console are the Elastic IP (Deploy step 1) and the Route 53 records (Deploy step 2) — and even those can be done from the CLI. Everything else is Terraform / CLI / SSH from your laptop. No IAM or other Console clicking.

All assets Terraform creates are automatically tagged to conform to the Graphwise AWS tagging requirements.

Kit Layout

Files marked `[git-ignored]` are never committed — they are credentials, licenses, or Terraform run artifacts you supply or generate locally. Everything else is tracked and ships in the kit.

terraform-subdomain/	(the kit root; copy this per stack)
├─ README.md	← this file
├─ main.tf	EC2 + EIP + security group + IAM
├─ variables.tf	all input variables + defaults + descriptions
├─ outputs.tf	instance_id, public_ip, DNS names
├─ versions.tf	Terraform + provider version pins
├─ user-data.sh.tpl	cloud-init: preps host + optionally builds stack
├─ terraform.tfvars.example	→ copy to terraform.tfvars, fill in your values
├─ terraform.tfvars	[git-ignored] your subdomain, EIP, key pair, CIDR
├─ graphwise-secrets.yaml	[git-ignored] maven, AWS Bedrock, n8n license/encryption
├─ n8n.txt	[git-ignored] AWS creds + extractor auth for workflow env
├─ .terraform/	[git-ignored] provider plugins (terraform init)
├─ .terraform.lock.hcl	[git-ignored] provider lock file
├─ terraform.tfstate	[git-ignored] live state after apply
├─ terraform.tfstate.backup	[git-ignored] previous state
├─ files/	
│ └─ n8n-pg-dumpall.sql.tar.gz	committed – known-good n8n workflow DB seed
│ └─ n8n-pg-dumpall.sql	[git-ignored] raw uncompressed dump (generated locally)
│ └─ licenses/	
│ └─ poolparty.key	[git-ignored] PoolParty license
│ └─ graphdb.license	[git-ignored] GraphDB EE license
│ └─ uv-license.key	[git-ignored] UnifiedViews license
└─ prep-scripts/	
│ └─ check-prereqs.sh	macOS preflight: verifies toolchain + AWS auth
│ └─ manage-stacks.sh	add/list/remove stack SSH entries in ~/.zprofile
│ └─ stack-scp.sh	scp wrapper using manage-stacks.sh key/host entries
│ └─ pull-config.sh	save live secrets + wildcard cert from EC2 to laptop
│ └─ push-config.sh	push saved secrets + cert to a fresh EC2

The first deployment step is to copy the entire `terraform-subdomain` folder to a new folder named for the stack (e.g. `terraform-kaizer`), edit three files (`terraform.tfvars`, `graphwise-secrets.yaml`, `n8n.txt`), then run a single command to build it out.

Kit Scripts

Run all scripts from your **per-stack Terraform folder** (e.g. `~/Desktop/terraform-kaizer/`), not from the git repo root.

`check-prereqs.sh` — Laptop preflight

Read-only. Checks and reports — never changes anything. Run this before anything else on a fresh laptop or after an OS upgrade.

What it checks: macOS version and CPU architecture, Homebrew, AWS CLI, Terraform, SSH, `dig`, `rsync`, AWS authentication (`aws sts get-caller-identity`), Python 3 + PyYAML, and IDE presence (informational).

```
cd ~/Desktop/terraform-kaiser
./prep-scripts/check-prereqs.sh
```

All checks print ✓ (green), ! (warning), or ✗ (error). Fix every ✗ before proceeding.

manage-stacks.sh — SSH multi-stack manager

Writes labelled blocks into `~/.zprofile` so you can SSH to any stack with a short alias. Each block exports `GW_KEY_<name>`, `GW_HOST_<name>`, and a configurable SSH alias. These variables are also read automatically by `stack-scp.sh`, `pull-config.sh`, and `push-config.sh`.

```
./prep-scripts/manage-stacks.sh           # interactive menu
./prep-scripts/manage-stacks.sh list      # print all configured stacks
./prep-scripts/manage-stacks.sh add       # add a new stack (interactive prompts)
./prep-scripts/manage-stacks.sh remove    # remove a stack (picker)
```

Adding a stack:

```
./prep-scripts/manage-stacks.sh add
# Stack name:      kaiser
# Key file path:   ~/.ssh/kaiser-stack-key.pem
# Hostname:        kaiser.gw-pse.com
# Alias name:      sshkaiser (press Enter for default)
```

After adding, open a new terminal or `source ~/.zprofile`, then:

```
sshkaiser    # → ssh -i ~/.ssh/kaiser-stack-key.pem ec2-user@kaiser.gw-pse.com
```

Listing all configured stacks:

```
./prep-scripts/manage-stacks.sh list
#  NAME      ALIAS      HOST                KEY
#  kaiser     sshkaiser   kaiser.gw-pse.com   ~/.ssh/kaiser-stack-key.pem
#  acme       sshacme     acme.gw-pse.com     ~/.ssh/acme-stack-key.pem
```

stack-scp.sh — Authenticated file transfer

`scp` wrapper that reads key and host from `~/.zprofile`. Prefix EC2-side paths with `:`.

```
./prep-scripts/stack-scp.sh [--stack <name>] [--r] <source> <dest>

# Push a file to the EC2
./prep-scripts/stack-scp.sh logo.png :~/logo.png

# Pull a file from the EC2
./prep-scripts/stack-scp.sh :~/wildcard-tls.yaml ./

# Recursive push
./prep-scripts/stack-scp.sh -r ./data :~/staging-data/

# Named stack, skip picker
./prep-scripts/stack-scp.sh --stack kaiser -r :~/gsb/files/ ./local-backup/
```

With no `--stack`, an interactive picker lets you choose; `--stack <name>` skips the picker.

pull-config.sh — Snapshot EC2 state to laptop

SSH to the EC2 and pull everything needed to survive a `terraform destroy`. **Run this before every destroy.**

What gets pulled:

- `graphwise-secrets.yaml` — rebuilt from the live Kubernetes Secrets the pods are consuming
- License files: `poolparty.key`, `graphdb.license`, `uv-license.key`
- `licenses/wildcard-tls.yaml` — the live Let's Encrypt wildcard cert (saves a rate-limit slot on rebuild)
- `graphwise-stack-chart-values.yaml` + diff (drift detector)
- `dashboard-kubeconfig.yaml`

Everything lands in a timestamped folder in the current directory:

```
graphwise-config-kaiser.gw-pse.com-20260708T143012Z/
  graphwise-secrets.yaml
  graphwise-stack-chart-values.yaml
  graphwise-stack-chart-values.diff    (only if drift vs. git baseline)
  dashboard-kubeconfig.yaml
  licenses/
    poolparty.key
    graphdb.license
    uv-license.key
    wildcard-tls.yaml
```

```
cd ~/Desktop/terraform-kaiser
./prep-scripts/pull-config.sh
```

If `manage-stacks.sh` was used, the script picks up `GW_KEY_*` / `GW_HOST_*` from `~/.zprofile` automatically. Otherwise pass them inline:

```
GRAPHWISE_KEY=~/.ssh/kaiser-stack-key.pem \  
GRAPHWISE_HOST=kaiser.gw-pse.com \  
./prep-scripts/pull-config.sh
```

push-config.sh — Restore snapshot to a fresh EC2

Sends the snapshot created by `pull-config.sh` back to a newly provisioned EC2. Run this **after** `terraform apply` but **before** `cluster-bootstrap.sh` on the EC2.

Snapshot file	EC2 destination
graphwise-secrets.yaml	~/graphwise-secrets.yaml
licenses/poolparty.key	~/gsb/files/licenses/
licenses/graphdb.license	~/gsb/files/licenses/
licenses/uv-license.key	~/gsb/files/licenses/
licenses/wildcard-tls.yaml	~/wildcard-tls-saved.yaml

The **n8n encryption key** is handled automatically: `push-config.sh` reads the fresh key Terraform wrote to the new EC2 and splices it into `graphwise-secrets.yaml` before pushing. The old key from the prior stack is not forwarded — the new database uses the new key.

```
cd ~/Desktop/terraform-kaiser  
./prep-scripts/push-config.sh                # auto-discovers most recent snapshot  
./prep-scripts/push-config.sh --list         # show available snapshots  
./prep-scripts/push-config.sh --snapshot ./graphwise-config-kaiser.gw-pse.com-20260708T143012Z  
./prep-scripts/push-config.sh --skip-cert    # force fresh LE issuance  
./prep-scripts/push-config.sh --skip-secrets # licenses + cert only  
./prep-scripts/push-config.sh --keep-local-encryption-key # don't splice the EC2's fresh n8n  
key
```

Laptop Prerequisites (one-time)

Your AWS PSE user account has already been created. You receive two CSV files with your Access Key ID and Secret Access Key.

macOS — install the toolchain:


```
# Install Homebrew (skip if brew --version already works)
/bin/bash -c "$(curl -fsSL
    https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

brew install awscli terraform      # ssh + dig ship with macOS
```

(Linux/Windows: install the same three tools with your package manager — `awscli`, `terraform`, and an SSH client.) Working the stack through an IDE (PyCharm, VS Code) is recommended — it makes running code, notebooks, and version control easy.

Verify the toolchain:

```
aws --version && terraform version && ssh -V && dig -v
```

Configure the AWS CLI with the CSV credentials, either interactively:

```
aws configure      # paste: Access Key ID, Secret, region us-west-2, output json
```

...or by writing the files directly:

```
mkdir -p ~/.aws && chmod 700 ~/.aws

cat > ~/.aws/credentials <<'EOF'
[default]
aws_access_key_id      = AKIA...your-key...
aws_secret_access_key = ...your-secret...
EOF

cat > ~/.aws/config <<'EOF'
[default]
region = us-west-2
output = json
EOF

chmod 600 ~/.aws/credentials ~/.aws/config
```

Verify it resolves to the right user:

```
aws sts get-caller-identity
```

Then confirm the full laptop toolchain is ready:

```
./prep-scripts/check-prereqs.sh
```

The output should come back clean; if not, remediate until it does.

ZSH Profile SSH Convenience

To make accessing multiple stacks easier, keep a set of exported variables and aliases in `$HOME/.zprofile` so you can SSH and SCP to any stack by name. The easiest way to manage these blocks is `manage-stacks.sh` (above), which writes and removes them for you:

```
./prep-scripts/manage-stacks.sh
```

After any manual edit to `.zprofile`, run `source ~/.zprofile` or restart the terminal session for the changes to take effect.

Deploy a New Stack

AWS region: Use `us-west-2` (Oregon). Other regions are not yet validated — future versions will address multi-region support.

Step 0 — Create the EC2 key pair (*laptop*)

Create your key pair and save the private `.pem` on your laptop — Terraform attaches the public key to the instance, and you SSH in with the `.pem`:

```
aws ec2 create-key-pair --region us-west-2 --key-name {lastname}-stack-key \
  --query 'KeyMaterial' --output text > ~/.ssh/{lastname}-stack-key.pem
chmod 400 ~/.ssh/{lastname}-stack-key.pem
```

Use the `{lastname}-stack-key` naming convention for `key_pair_name` in `terraform.tfvars` (Step 4) and the `.pem` path everywhere below. One key pair can be reused across all stacks, or create one per stack.

Step 1 — Allocate an Elastic IP (*laptop*)

Each stack needs a static public IP. Allocate one per stack you intend to maintain.

```
aws ec2 allocate-address --domain vpc --region us-west-2
# Save BOTH "AllocationId" (eipalloc-...) and "PublicIp" (<eip>) from the output.
```

The EIP survives `terraform destroy / apply` cycles, so DNS never needs re-pointing. If you permanently decommission a stack, release the EIP and remove the DNS records to stop charges.

Step 2 — Add DNS records (*laptop / console*)

Shared Route 53 zone: `gw-pse.com` — hosted zone ID `Z03476331NSJ3X9EZAGNE`. **Do not change this value in `terraform.tfvars`.**

Add two A records, both pointing at `<eip>` :

Name	Type	Value
<code><sub>.gw-pse.com</code>	A	<code><eip></code>
<code>*.<sub>.gw-pse.com</code>	A	<code><eip></code>

Via CLI (`UPSERT` = create-or-update, safe to re-run):

```
SUB=<sub>
EIP=<eip>
ZONE_ID=$(aws route53 list-hosted-zones \
  --query "HostedZones[?Name=='gw-pse.com.'].Id" --output text | sed 's|/hostedzone/||')

aws route53 change-resource-record-sets --hosted-zone-id "$ZONE_ID" --change-batch "$(cat
  <<JSON
{
  "Comment": "graphwise stack $SUB",
  "Changes": [
    { "Action": "UPSERT", "ResourceRecordSet": {
      "Name": "$SUB.gw-pse.com", "Type": "A", "TTL": 300,
      "ResourceRecords": [ { "Value": "$EIP" } ] } },
    { "Action": "UPSERT", "ResourceRecordSet": {
      "Name": ".*$SUB.gw-pse.com", "Type": "A", "TTL": 300,
      "ResourceRecords": [ { "Value": "$EIP" } ] } }
  ]
}
JSON
)"
```

Verify propagation:

```
dig +short <sub>.gw-pse.com poolparty.<sub>.gw-pse.com # both must print <eip>
```

DNS must be live **before** Terraform runs (Step 7) — cert-manager uses it for the wildcard certificate.

Step 3 — Create the stack working folder (*laptop*)

Copy the kit folder, naming it for this stack:

```
cp -r terraform-subdomain ~/Desktop/terraform-<sub>
cd ~/Desktop/terraform-<sub>
```

This folder is the home for everything about this stack: Terraform state, credentials, snapshots, and scripts. Run all subsequent steps from here.

Step 4 — Edit `terraform.tfvars` (laptop)

```
cp terraform.tfvars.example terraform.tfvars
```

```
region                = "us-west-2"
base_domain            = "gw-pse.com"
subdomain              = "<sub>"
route53_zone_id       = "Z03476331NSJ3X9EZAGNE"
le_email              = "you@graphwise.ai"
key_pair_name          = "{lastname}-stack-key"      # name only, no .pem
admin_cidr             = "<your-ip>/32"              # curl -4 icanhazip.com
availability_zone      = "us-west-2a"
existing_eip_allocation_id = "eipalloc-..."        # from Step 1
creator               = "Your Name"
purpose               = "Customer X presales demo"
```

Step 5 — Fill in `graphwise-secrets.yaml` (laptop)

The kit ships this file partially pre-filled with internal-use credentials. Open it and confirm or update as needed:

```
maven:
  user: <maven-username>      # provided in the kit
  pass: <maven-password>      # provided in the kit

graphrag-secrets:
  awsCredentials:
    region: us-west-2
    accessKeyId: AKIA...      # your AWS access key ID (from your CSV)
    secretAccessKey: ...      # your AWS secret access key

  n8nLicense:
    activationKey: ...         # provided in the kit

  n8nEncryption:
    key: ...                   # provided in the kit — DO NOT CHANGE
```

The `n8nEncryption.key` must remain constant across rebuilds of the same subdomain. It is the key that encrypts all n8n credentials in the database. Changing it makes the existing database unreadable. The kit's value is correct — leave it alone. `push-config.sh` handles splicing the right key automatically on rebuild.

Step 6 — Prepare `n8n.txt` (laptop, for later use on EC2)

`n8n.txt` holds the per-deployment credentials injected into the n8n workflow pod environment. You write this file **on the EC2** during POC setup (Chapter 2, Step 3.2), but it helps to have the values ready now.

```
AWS_ACCESS_KEY_ID=AKIA...      # your AWS access key (same as graphwise-secrets.yaml)
AWS_SECRET_ACCESS_KEY=...      # your AWS secret access key
EXTRACTOR_AUTH=Basic <base64>  # Graph Modeling superadmin credentials (see note)
EXTRACTOR_PROJECT_ID=<uuid>     # Graph Modeling project UUID – set after Step 2 in Chapter 2
```

Generate `EXTRACTOR_AUTH` using `printf` (not `echo` — `echo` adds a trailing newline that causes 401 errors):

```
printf 'superadmin:corgiDAD#2' | base64
# Prefix the output: EXTRACTOR_AUTH=Basic <output>
```

`EXTRACTOR_PROJECT_ID` is the UUID of the Graph Modeling project you create in Chapter 2, Step 2. You cannot fill it in until that project exists. Do **not** change `EXTRACTOR_AUTH` unless the superadmin password changes.

Step 7 — Provision the EC2 (*laptop*)

You **must** be in the subdomain folder to run this. Each subdomain is independent, so running multiple stacks is supported.

```
cd ~/Desktop/terraform-<sub>
terraform init
terraform plan
terraform apply
```

Terraform creates the EC2, attaches the EIP, and starts cloud-init. The stack build continues automatically on the EC2. Monitor it:

```
ssh -i ~/.ssh/{lastname}-stack-key.pem ec2-user@<sub>.gw-pse.com
sudo tail -f /var/log/bootstrap.log
```

If this is a rebuild of an existing subdomain and you ran `pull-config.sh` before destroying, **now is the time to run `push-config.sh`** from your laptop (before proceeding on the EC2). This restores the saved wildcard cert, avoiding a new Let's Encrypt issuance.

Step 8 — Build the stack over SSH (*EC2*)

Wait for cloud-init to finish (check with `kubectl get nodes`), then run:

Option A — Single script (after you have experience):

```
cd ~/gsb
./prep-scripts/deploy-stack.sh <sub> gw-pse.com
```

Option B — Step by step (recommended until familiar):

```
cd ~/gsb

./prep-scripts/cluster-bootstrap.sh
# Wait: kubectl get pods -A - all pods running before continuing

./prep-scripts/validate-bootstrap.sh
./prep-scripts/extract-poolparty-realm.sh
./prep-scripts/preflight-reset-helm.sh
./prep-scripts/reset-helm.sh --yes <sub> gw-pse.com
# Wait: kubectl get pods -A - all 1/1 Running before continuing

./prep-scripts/restore-n8n-dumpall.sh
./prep-scripts/validate-stack.sh
```

Monitor pod status:

```
kubectl get pods -A      # one-shot
kubectl get pods -A -w    # live watch (Ctrl-C to exit)
```

Typical build time: 15–20 minutes. Graph Modeling (PoolParty) is always the last pod to reach Running status.

Step 9 — Verify the stack (EC2)

```
export APEX=<sub>.gw-pse.com
for h in $APEX poolparty.$APEX auth.$APEX graphdb.$APEX graphrag.$APEX; do
  printf '%-45s ' "$h"
  curl -s -o /dev/null -w 'http=%{http_code}\n' "https://$h/" --max-time 10
done
```

All services should return `http=200` or `http=302` . Then browse `https://<sub>.gw-pse.com` — the console landing page.

Step 10 — Add customer logo (optional) (laptop + EC2)

```
# From laptop — upload the PNG
scp -i $GRAPHWISE_KEY logo.png $GRAPHWISE_USER@$GRAPHWISE_HOST:~/logo.png

# From EC2 — set the logo
cd ~/gsb
./prep-scripts/set-logo.sh
```

Use a PNG with a transparent background — the console header is blue.

After the Build: Pull Config

Run this immediately after every successful stack build. Let's Encrypt limits wildcard certificate issuance to 5 per week per domain. Saving the cert from the current build means the next rebuild reuses it, preserving your rate-limit allocation.

```
cd ~/Desktop/terraform-<sub>
./prep-scripts/pull-config.sh
```

Add the stack to your SSH manager for convenient access going forward:

```
./prep-scripts/manage-stacks.sh add
```

Rebuild / Destroy-and-Rebuild Flow

```
# 1. Before destroying – snapshot secrets + cert
cd ~/Desktop/terraform-<sub>
./prep-scripts/pull-config.sh

# 2. Destroy
terraform destroy

# 3. Rebuild
terraform apply

# 4. Restore snapshot before bootstrap (from laptop)
./prep-scripts/push-config.sh

# 5. SSH to EC2 and continue the build
sshkaiser
# → cluster-bootstrap.sh → reset-helm.sh → restore-n8n-dumpall.sh → validate-stack.sh
```

The wildcard cert saved in step 1 is restored in step 4, so cert-manager sees a valid cert already in place and skips the LE DNS-01 round trip. This is why `pull-config.sh` before destroy is so important.

Auto-Shutdown and Restart

Each stack has a CloudWatch alarm that **stops the EC2** when CPU utilization stays below 5% for 8 consecutive hours. This guards against forgotten idle stacks accumulating charges.

To restart: use the AWS Console (EC2 → Instances → Start) or:

```
aws ec2 start-instances --region us-west-2 --instance-ids <instance-id>
```

On boot, a systemd unit (`graphwise-cluster-resume.service`) runs automatically and restores all workloads — no manual intervention needed. Allow ~5 minutes for all pods to reach Running status.

Disable auto-shutdown during demo days to prevent an idle window between sessions from triggering a stop: set `auto_shutdown_enabled = false` in `terraform.tfvars` and `terraform apply` .

Loading Staging Data (*laptop*)

The EC2's `~/staging-data/` folder is mounted into the KIND cluster as a PVC, making it available to pods. Use it to stage data files for ingest workflows.

```
scp -r -i $GRAPHWISE_KEY {source_folder} $GRAPHWISE_USER@$GRAPHWISE_HOST:~/staging-data/
```

After uploading, fix permissions so the n8n pod (different UID) can read and write:

```
# On the EC2
chmod -R a+rwX ~/staging-data/
```

Stack Tear Down (*laptop*)

`terraform destroy` deletes the EC2 and its EBS volume — **all stack data is lost** (PoolParty projects, GraphDB repos, Keycloak users, n8n workflows, ingested documents). The EIP and DNS records survive because they are pre-allocated.

Always run `pull-config.sh` before destroying if you plan to rebuild the same subdomain — otherwise you burn a weekly Let's Encrypt allocation.

```
cd ~/Desktop/terraform-<sub>
./prep-scripts/pull-config.sh      # save secrets + cert
terraform destroy
```

If the stack is being permanently decommissioned: release the EIP and delete the two Route 53 A records to stop charges.

Deployment Troubleshooting

Pod status

```
kubectl get pods -A           # one-shot snapshot
kubectl get pods -A -w        # live watch (Ctrl-C to exit)
kubectl get pods -n graphwise
kubectl get pods -n graphrag
kubectl get pods -n keycloak
kubectl get pods -n cert-manager
```

Bootstrap log

```
ssh -i $GRAPHWISE_KEY $GRAPHWISE_USER@$GRAPHWISE_HOST
sudo tail -f /var/log/bootstrap.log
```

Pod logs

```
kubectl logs -n <namespace> <pod-name>
kubectl logs -n <namespace> <pod-name> -f           # stream live
kubectl logs -n <namespace> <pod-name> --tail=100
kubectl logs -n <namespace> <pod-name> -c <container-name> # multi-container pod
kubectl logs -n <namespace> <pod-name> --previous      # after crash-loop
```

Find the pod name by app:

```
kubectl get pods -n graphwise | grep poolparty
```

Kubernetes events

Often more useful than logs for startup failures:

```
kubectl get events -n <namespace> --sort-by='.lastTimestamp'
```

Events explain why a pod is `Pending`, `CrashLoopBackOff`, or stuck in `Init`.

Orphaned IAM role after a manual delete (*laptop / console*)

Always tear down with `terraform destroy`. If a stack was instead deleted manually — terminating the EC2 in the Console, or deleting resources with the CLI — the stack's IAM role and instance profile are usually left behind, and deleting the role fails with:

Cannot delete entity, must remove roles from instance profile first.

Terraform creates the role *inside* an instance profile (both named `graphwise-stack-<subdomain>-ec2-*`), and IAM refuses to delete a role still listed in a profile. Clean up in this order:

```
SUB=<subdomain>    # e.g. kaiser

# 1. Confirm nothing is still using the profile (must print nothing)
aws ec2 describe-instances \
  --filters "Name=iam-instance-profile.arn,Values=arn:aws:iam::<account-id>:instance-
    profile/graphwise-stack-{$SUB}-ec2-profile" \
  --query 'Reservations[].Instances[].[InstanceId,State.Name]' --output text

# 2. Delete any inline policies still on the role (e.g. the Route 53 policy)
aws iam list-role-policies --role-name graphwise-stack-{$SUB}-ec2-role --output text
aws iam delete-role-policy --role-name graphwise-stack-{$SUB}-ec2-role --policy-name <name-
  from-above>

# 3. Detach the role from the profile, then delete both
aws iam remove-role-from-instance-profile \
  --instance-profile-name graphwise-stack-{$SUB}-ec2-profile \
  --role-name graphwise-stack-{$SUB}-ec2-role
aws iam delete-role --role-name graphwise-stack-{$SUB}-ec2-role
aws iam delete-instance-profile --instance-profile-name graphwise-stack-{$SUB}-ec2-profile
```

Instance profiles are not exposed in the Console, so step 3 must be done with the CLI — this is exactly why the Console's "Delete role" button fails on these roles.

Terraform state is now out of sync. The `terraform.tfstate` in that subdomain's folder still records the role, profile, EC2, and security group. Before the next `terraform apply`, run `terraform destroy` anyway (it reconciles already-gone resources away) or remove the stale entries with `terraform state rm`. Also sweep for other manual-delete leftovers: the security group, the EIP *association* (the EIP itself is pre-allocated and should stay), and the Route 53 records.

TLS certificate issues — forcing Let's Encrypt to re-issue

Let's Encrypt limits wildcard cert issuance to **5 per week per domain**. If you hit the limit or ended up with a staging cert, delete the wildcard TLS Secret — cert-manager detects it is gone and immediately requests a fresh production cert.

THINK TWICE before reissuing — you have only 5 per week.

```
kubectl delete secret wildcard-tls -n cert-manager
```

Kubernetes Reflector auto-removes the mirrored copies in `graphwise`, `graphdb`, `graphrag`, `keycloak`, and `monitoring`, then repopulates them once the new cert arrives (~2–3 minutes via DNS-01 / Route 53).

Monitor progress:

```
kubectl get certificate wildcard-tls -n cert-manager -w
kubectl get order,challenge -n cert-manager
kubectl describe order -n cert-manager | tail -30
kubectl logs -n cert-manager deploy/cert-manager --tail=40 2>&1 | grep -iE
'error|denied|challenge|dns|present'
```

If the ClusterIssuer itself was pointed at the LE staging server, reset it first, then delete the Secret:

```
kubectl patch clusterissuer letsencrypt-prod \
  --type=merge \
  -p '{"spec":{"acme":{"server":"https://acme-v02.api.letsencrypt.org/directory"}}}'

kubectl delete secret wildcard-tls -n cert-manager
```

Why staging certs break the stack: PoolParty's JVM validates the full TLS chain when it connects to Keycloak at startup. The Let's Encrypt staging chain is not trusted by the JVM's default trust store, so the OIDC handshake fails and PoolParty never reaches Ready. Always use `letsencrypt-prod`.

Kubernetes Dashboard and Grafana

Kubernetes Dashboard

The `dashboard-kubeconfig.yaml` file in your `pull-config.sh` snapshot is the authentication file for the Kubernetes Dashboard. When you open the dashboard from the console landing page, choose the **file** method and upload this file.

Each stack generates its own bearer token — the kubeconfig from one stack does not work on another.

Grafana

Access Grafana from the console landing page; credentials are shown on the panel. The underlying Prometheus collects full cluster metrics from the KIND stack. Graphwise-specific dashboard recipes (GraphDB query latency, PoolParty heap, etc.) are planned for a future release.

Chapter 2 — POC Configuration

POC Overview

Chapter 2 assumes the stack is already running (Chapter 1 complete, `validate-stack.sh` passing). The steps here load data, configure the workflow engine, run the ingestion pipelines, and validate the knowledge graph before activating the chatbot.

The overall POC arc:

Step	What	Tool
1	Load GraphDB (ontology + taxonomy + schema)	GraphDB Workbench
2	Load Graph Modeling (ontology + taxonomy)	Graph Modeling (PoolParty)
3	Configure the stack n8n	stack n8n env + UI
4	Structured ingest (CSV → typed graph nodes)	n8n <i>Structured Ingest</i>
5	Unstructured ingest (docs → chunks → embeddings)	n8n <i>Unstructured Ingest</i>
6	Annotate both datasets (one run)	n8n <i>Extractor</i>
7	Validate structured graph	<code>Structured-Validate-GraphDB.ipynb</code>
8	Validate unstructured graph + GraphRAG pre-flight	<code>Unstructured-Validate-GraphDB.ipynb</code>
9	Validate GraphRAG retrieval	<code>GraphRAG-Prompt-Validation.ipynb</code>
10	Activate the chatbot (<i>optional</i>)	n8n seed + <code>Configuration</code>
11	Prune n8n execution history (<i>maintenance</i>)	n8n <i>N8N Prune Database</i>

The workflow engine (n8n) is the most operationally complex component — it spans multiple steps and has a lifecycle that is easy to misunderstand. Read the [n8n lifecycle box](#) at the start of Step 3 before touching anything n8n-related.

Step 1 — Load GraphDB

Create the repository. In the GraphDB Workbench, go to **Setup** → **Repositories** → **Create new repository** → **GraphDB Repository**:

- **Repository ID:** `<project-repo-id>` (e.g. `coverage`, `va-benefits`)

- **Ruleset: OWL-Max (Optimized)** — the reasoner the ontology/taxonomy rely on
- **Supports context index:**  enabled — required for named-graph imports
- **Enable full-text search (NLP):**  enabled

Make it the **active repository** (click the pin/connect icon) before importing. Enable **Setup** → **Autocomplete** for the repository — GraphDB's MCP server requires it.

Import in order (enter any named-graph URI **exactly** as shown, trailing slash included):

#	File	Target graph
1	modeling/ontology.ttl	default graph
2	modeling/taxonomy.ttl	default graph
3	modeling/schema.ttl	default graph

The **structured** data (typed nodes) is loaded by n8n in Step 4 and the **chunk** nodes in Step 5; their concept tags are written by the **Extractor** in Step 6 — do **not** import those manually. If your dataset ships an external vocabulary (e.g. UMLS), import it into its own named graph per the dataset's modeling notes.

Step 2 — Load Graph Modeling

First-time login. Sign in with `superadmin` / `poolparty` . Set the password to `corgiDAD#2` when prompted (use exactly this value — the rest of the runbook and stack config assume it).

Create the project. Create a new project named for your POC and **note its UUID** — you need it for `EXTRACTOR_PROJECT_ID` in `n8n.txt` (Step 3.2) and for `poolPartyProjectId` in the Configuration node (Step 3.7).

Import in order:

#	File	Import type	Notes
1	modeling/ontology.ttl	Ontology	OWL class definitions
2	modeling/taxonomy.ttl	Thesaurus	SKOS vocabulary
3	modeling/schema.ttl	Project container	wires OWL classes to taxonomy concepts (set <code>ppt:BaseUrl</code> to your server first)

Then: **Corpus** → **Rebuild Extraction Model** and wait for it to complete before running the Extractor.

Don't skip the rebuild on a new project. Importing the thesaurus does *not* populate the extractor's concept index. Until you Rebuild Extraction Model, every Extractor call fails with `HTTP 400: Concept Index is empty for projectId ...`, and the Extractor's `Aggregate` guard aborts before touching the graph. Also confirm you imported the thesaurus into the project whose UUID you put in `EXTRACTOR_PROJECT_ID`.

Connect Graph Modeling to GraphDB. Go to **Systems** → **Graph Databases** → **GraphDB** → **Create** and set the URL:

```
https://graphdb-projects.<sub>.gw-pse.com/repositories/<project-repo-id>
```

Step 3 — Configure n8n

n8n Lifecycle — Three Things to Understand

Before touching anything n8n-related, understand these three distinct phases. Mixing them up is the main source of confusion.

Phase 1 — Database restore (stack build, already done). `restore-n8n-dumpall.sh` was run in Chapter 1, Step 8. It wiped the blank database and loaded the known-good SQL seed, which contains all ~31 pre-built workflows plus the credential slots (filled with placeholder values). This happens once per stack build. The restore produces a working n8n install with workflows intact. **You do not re-run this** unless you are resetting to a clean baseline.

Phase 2 — Encryption key (managed automatically). Every n8n instance encrypts its stored credentials with `N8N_ENCRYPTION_KEY`. This key is generated by Terraform at EC2 creation and written into the Kubernetes Secret. **It must stay constant for the lifetime of the database** — if it changes, all stored credentials become unreadable garbage. `push-config.sh` handles this automatically during rebuilds. Just do not manually change `N8N_ENCRYPTION_KEY` anywhere.

Phase 3 — Per-build and per-POC configuration (you do this now). After every restore, four things need to be done before n8n workflows will run:

1. **Kubernetes Secret** — write `~/n8n.txt` on the EC2 and create `n8n-poc-creds`
2. **Environment injection** — patch the n8n Deployment with the Secret's values + behavior flags
3. **API key rotation** — the JWT stored in the database is stale after restore; generate a new one
4. **Configuration node** — update the deployment-specific URLs/IDs in the `Configuration` workflow

These steps are detailed below.

3.1 — Restore the n8n database

Already done during the stack build (`restore-n8n-dumpall.sh`). Confirm the pod is up:

```
kubectl -n graphrag rollout status deploy/graphrag-workflows
```

If you need to reset n8n to the clean seed at any point:

```
cd ~/gsb
./prep-scripts/restore-n8n-dumpall.sh
kubectl -n graphrag rollout status deploy/graphrag-workflows
```

3.2 — Write `~/n8n.txt` and create the Kubernetes Secret (EC2)

Now that you have the Graph Modeling project UUID from Step 2, write `~/n8n.txt` on the **EC2** (one `KEY=VALUE` per line — no quotes, no trailing spaces; `--from-env-file` takes everything after `=` literally):

```
cat > ~/n8n.txt <<'EOF'
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
EXTRACTOR_AUTH=Basic <base64-of-superadmin:corgiDAD#2>
EXTRACTOR_PROJECT_ID=<your-graph-modeling-project-uuid>
EOF
```

Generate `EXTRACTOR_AUTH` (use `printf`, not `echo`):

```
printf 'superadmin:corgiDAD#2' | base64
```

Create the Kubernetes Secret:

```
kubectl -n graphrag create secret generic n8n-poc-creds --from-env-file="$HOME/n8n.txt"
```

If the Secret already exists from a prior run:

```
kubectl -n graphrag delete secret n8n-poc-creds
kubectl -n graphrag create secret generic n8n-poc-creds --from-env-file="$HOME/n8n.txt"
```

After a **terraform destroy** and **rebuild**, the Secret is gone (it lived in the cluster). You must re-create it from `~/n8n.txt` every time. Keep `~/n8n.txt` on the EC2 or re-push it with `stack-scp.sh`.

3.3 — Inject environment variables into the n8n Deployment (EC2)

Two commands — run both:

```
# Inject the credentials from the Secret
kubectl -n graphrag set env deploy/graphrag-workflows --from=secret/n8n-poc-creds

# Inject behavior flags that allow workflow code nodes to function
kubectl -n graphrag set env deploy/graphrag-workflows \
  N8N_BLOCK_ENV_ACCESS_IN_NODE=false \
  NODE_FUNCTION_ALLOW_BUILTIN='*' \
  NODE_FUNCTION_ALLOW_EXTERNAL=js-tiktoken \
  N8N_ALLOW_CODE_NODE_EXTERNAL_FILES=true \
  N8N_RUNNERS_TASK_TIMEOUT=1800 \
  N8N_RUNNERS_HEARTBEAT_INTERVAL=600

kubectl -n graphrag rollout status deploy/graphrag-workflows
```

Flag	Why
N8N_BLOCK_ENV_ACCESS_IN_NODE=false	lets nodes read <code>{{ \$env.* }}</code> ; without it the Configuration node fails
NODE_FUNCTION_ALLOW_BUILTIN=*	Code nodes may <code>require()</code> Node built-ins
NODE_FUNCTION_ALLOW_EXTERNAL=js-tiktoken	Unstructured Ingest's token-accurate chunker
N8N_ALLOW_CODE_NODE_EXTERNAL_FILES=true	Code-node file access
N8N_RUNNERS_TASK_TIMEOUT=1800	long corpus runs (~30 min) do not get killed
N8N_RUNNERS_HEARTBEAT_INTERVAL=600	headroom for large documents

These patches are wiped by `helm upgrade`. Re-run both `kubectl set env` commands any time `reset-helm.sh` is run.

3.4 — Log in to the n8n UI

Open `https://graphrag.<sub>.gw-pse.com` (also reachable as `workflows.<sub>.gw-pse.com`) and log in:

- **Email:** `kent.stroker@graphwise.ai`
- **Password:** `graphDB#1`

Confirm the workflows list is populated (~31 workflows) and the `Main` workflow exists and shows as active.

3.5 — Rotate the API key (n8n UI)

The database restore brings in a JWT that was valid on the source instance but is invalid here. It must be rotated before any workflow can make internal API calls. **The same rotated key is used in two credentials** — generate it once, then wire it into both.

Generate the new key:

1. n8n UI → **Settings** → **API** → delete the existing `graphwise-graphrag` key
2. Create a new key named `graphwise-graphrag` — **copy the value immediately** (shown only once)

Update the `API_KEYS` Data Table — the step most commonly missed:

n8n UI → **Data Tables** → **API_KEYS** → open the `N8N` row → paste the new key into `value` → save.

The data table is what workflow nodes read at runtime — not the credential objects.
Skipping this step leaves the old (now-invalid) JWT in the table, causing every internal API call to return 401, silently in some nodes.

Create or update the `graphwise-graphrag` credential (used by most workflow HTTP Request nodes) —

n8n UI → **Credentials** → **New** → **n8n API**:

Field	Value
Name	<code>graphwise-graphrag</code>
API Key	the value copied above
Base URL	<code>http://graphrag-workflows:5678</code>

Create or update the `n8n Internals` credential (used by the N8N Prune Database workflow — the node type does not append `/api/v1` itself, so the Base URL must include it) — n8n UI → **Credentials** → **New** → **n8n API**:

Field	Value
Name	<code>n8n Internals</code>
API Key	the same value copied above
Base URL	<code>http://graphrag-workflows:5678/api/v1</code>

Both credentials use the same API key — the only difference is the Base URL suffix. If you later rotate the key again (e.g. after another restore), update the key in both credentials and in the `API_KEYS` data table row.

3.6 — Fill n8n credentials (*n8n UI*)

Go to n8n UI → **Credentials**. After a restore with a constant encryption key the blobs may already be populated — verify first and skip what is already filled.

Main LLM Model credentials (type: AWS):

Field	Value
Access Key ID	<code>AWS_ACCESS_KEY_ID</code> from <code>~/n8n.txt</code>
Secret Access Key	<code>AWS_SECRET_ACCESS_KEY</code> from <code>~/n8n.txt</code>
Region	<code>us-west-2</code>

PoolParty Credentials (type: HTTP Basic Auth):

Field	Value
Username	<code>superadmin</code>
Password	<code>corgiDAD#2</code>

Keycloak clientId / clientSecret (type: HTTP Basic Auth):

Field	Value
Username	<code>conversation-api-client</code>
Password	Keycloak admin → realm <code>graphrag</code> → Clients → <code>conversation-api-client</code> → Credentials tab

The Keycloak client secret is **generated per stack build** — it differs each time. Always retrieve it from Keycloak admin. If wrong, the JWT token verification sub-workflow returns `{"active": false}` and the chatbot silently rejects every request.

3.7 — Edit the Configuration node (*n8n UI*)

Open the **Configuration** workflow from the workflows list, then open its **Code node**. This object drives the chatbot and conversation flows and is called via Execute Workflow by `Main` and `Parallel - Main`. The three ingest workflows (Structured, Unstructured, Extractor) have their own separate Config nodes — those are edited in Steps 4–6.

Fields to change (infrastructure-specific):

Field	Seed value (example)	Set to
graphDBMcpUrl	https://graphdb-projects.kaiser.gw-pse.com/mcp	https://graphdb-projects.<sub>.gw-pse.com/mcp
graphDBMcpRepository	"coverage"	your GraphDB repository name
keycloakUrl	https://auth.kaiser.gw-pse.com	https://auth.<sub>.gw-pse.com
poolPartyServerUrl	https://poolparty.kaiser.gw-pse.com	https://poolparty.<sub>.gw-pse.com
poolPartyProjectId	"20670b0b-..."	your PoolParty project UUID (from Step 2)

Fields to confirm match your POC (must be consistent with ingest workflows):

Field	Default	Notes
vectorIndex	"doc-chunks"	must match <code>esIndex</code> in Unstructured Ingest Config
embeddingsModelId	"amazon.titan-embed-text-v2:0"	must byte-match <code>embedModelId</code> in Unstructured Ingest — KNN fails if they differ
mcpLLMModel	"us.anthropic.claude-sonnet-4-6"	Bedrock inference profile ID
primaryLLMModel	"us.anthropic.claude-sonnet-4-6"	Bedrock inference profile ID
secondaryLLMModel	"us.anthropic.claude-sonnet-4-6"	Bedrock inference profile ID

Stable — do NOT change (internal k8s service names, same across all deployments):

- `backendUrl` : http://graphrag-conversation:8080
- `graphRagComponentsUrl` : http://graphrag-components:8080
- `internalN8nUrl` : http://graphrag-workflows:5678
- `embeddingsProvider` : "aws" (change only if switching to OpenAI)
- `vectorStorePreset` : "elasticsearch_native"
- `graphDBMcpOntologyGraph` : "" — Step 1 loads `ontology.ttl` into the default graph, and empty = the default graph.

MCP transport: `graphDBMcpUrl` uses the **HTTP Streamable** transport. The deprecated `/mcp/sse` SSE path returns 406 — also set the MCP Client node's Server Transport to "HTTP Streamable". Save the workflow after editing.

3.8 — Stage data files (*laptop*)

Copy the `output/` tree to the EC2:

```
scp -r -i "$GRAPHWISE_KEY" output "$GRAPHWISE_USER@$GRAPHWISE_HOST:~/staging-data/"
```

Fix permissions on the EC2 so the n8n pod (different UID) can read:

```
chmod -R a+rwX ~/staging-data/output
```

3.9 — Mount the staging PVC into the n8n pod (*EC2*)

```
CNAME=$(kubectl -n graphrag get deploy graphrag-workflows \
-o jsonpath='{.spec.template.spec.containers[0].name}')

kubectl -n graphrag patch deploy graphrag-workflows --type=strategic -p \
'{"spec":{"template":{"spec":{"volumes":[{"name":"staging","persistentVolumeClaim":{"claimName":"staging-data"}]}},
"containers":[{"name":"$CNAME","volumeMounts":[{"name":"staging","mountPath":"/data/staging"}]}}}}'

kubectl -n graphrag rollout status deploy/graphrag-workflows

# Confirm the files are visible
kubectl -n graphrag exec deploy/graphrag-workflows -- ls /data/staging/output/csv | head
```

This mount is also wiped by `helm upgrade`. Re-apply the patch if `reset-helm.sh` is run. If `staging-data` PVC is missing, it wasn't created in the graphrag namespace — that is a stack/Helm concern.

Step 4 — Structured Ingest (*n8n*)

Note: The seeded workflow is configured for a specific prior dataset. The ontology namespace, graph URIs, CSV column mappings, SPARQL class names, and expected record counts will all need adjusting to match your dataset's structure. Treat the seed as a working template, not a ready-to-run workflow.

In the stack n8n, **Import from File** `n8n-workflows/Structured Ingest.json`.

Config node (Set node) — edit these fields:

Field	Seed value	Set to
graphdb	<code>https://graphdb-projects.<ref>.gw-pse.com/repositories/<ref-repo>/statements</code>	<code>https://graphdb-projects.<sub>.gw-pse.com/repositories/<repo-id>/statements</code>
poolparty	<code>https://poolparty.<ref>.gw-pse.com/extractor/api/tag</code>	<code>https://poolparty.<sub>.gw-pse.com/extractor/api/tag</code>
annotationGraph	reference annotation graph URI	your project's annotation graph URI
csvDir	<code>/data/staging/output/csv</code>	adjust only if your staging path differs
rdfDir	<code>/data/staging/output/rdf</code>	adjust only if your staging path differs
prefixes block	reference namespace	your project's ontology namespace URIs

`extractorProjectId` and `extractorAuth` are already `{{ $env.EXTRACTOR_PROJECT_ID }}` / `{{ $env.EXTRACTOR_AUTH }}` env var references — no change needed.

Verify: SPARQL Counts node (Code node) — edit if your ontology differs:

This node runs SPARQL COUNT queries to assert minimum record thresholds after ingest. The class names, PREFIX declaration, and expected counts are all dataset-specific — replace with classes from your ontology and appropriate counts.

Execute the workflow. It converts CSVs into typed graph nodes and loads them into GraphDB. Concept tagging happens later, in Step 6.

Step 5 — Unstructured Ingest (*n8n*)

Note: The seeded workflow is configured for a specific prior dataset. The chunk graph URI, ontology namespace, Elasticsearch index name, and embedding model must all match your project's choices — and must stay consistent with the Extractor workflow (Step 6) and the Configuration node (Step 3.7). Review every Config field before running.

Import from File `n8n-workflows/Unstructured Ingest.json`.

Config node (Set node) — edit these fields:

Field	Seed value	Set to
graphdb	<code>https://graphdb-projects.<ref>.aws- secretsmanager.amazonaws.com/repositories/<ref- repo>/statements</code>	<code>https://graphdb-projects.<sub>.aws- secretsmanager.amazonaws.com/repositories/<repo- id>/statements</code>
esIndex	<code>"doc-chunks"</code>	keep or choose your own name — must match <code>vectorIndex</code> in Configuration (Step 3.7)
embedModelId	<code>"amazon.titan-embed-text-v2:0"</code>	your Bedrock embedding model — must byte-match <code>embeddingsModelId</code> in Configuration
embedRegion	<code>"us-west-2"</code>	your AWS region
chunksGraph	reference chunks graph URI	your chunk graph URI — must match <code>chunksGraph</code> in Extractor Config (Step 6)
prefixes block	reference namespace	your project's ontology namespace URIs

`awsAccessKeyId` and `awsSecretAccessKey` are already env var references — no change needed. `esUrl` (`http://graphwise-stack-poolparty-elasticsearch.graphwise:9200`) is the in-cluster Elasticsearch service — **stable across all deployments, do not change**.

Execute the workflow. It reads the staged corpus, chunks each document, embeds with AWS Bedrock Titan v2, writes vectors to Elasticsearch, and writes chunk nodes into GraphDB.

- If the Elasticsearch index already exists from a prior run with a different mapping, delete it first: `curl -X DELETE http://graphwise-stack-poolparty-elasticsearch.graphwise:9200/<esIndex>` (run from inside a pod, or via `kubectl port-forward`) so the current mapping takes effect — `Create: ES Index` no-ops on an existing index.
- Embeddings are cached to `output/embeddings-cache.json` — re-runs reuse cached vectors for unchanged text (no time, no cost).
- The `Limit (TEST: ...)` nodes are **disabled**, so the full corpus is processed. Re-enable them (press `D` on each) for a quick test slice.
- If you get `ENOENT ... /data/staging/output/...`, the staging PVC is not mounted — redo Step 3.9.

Step 6 — Annotate (Extractor) (n8n)

Note: The seeded workflow is configured for a specific prior dataset. The annotation graph URIs, ontology namespace, chunk graph URI, and PoolParty concept base URI are all dataset-specific — and unlike the other workflows, several of these are hardcoded directly in Code and HTTP Request nodes rather than in the Config node alone. Review and update all six nodes below before running.

Import from File `n8n-workflows/Extractor.json`.

This workflow has hardcoded values in six different nodes. Edit all of them before running — partial edits leave stale URIs in the graph.

Node 1 — Config (Set node):

Field	Seed value	Set to
<code>graphdb</code>	<code>https://graphdb-projects.<ref>.gw-pse.com/repositories/<ref-repo>/statements</code>	<code>https://graphdb-projects.<sub>.gw-pse.com/repositories/<repo-id>/statements</code>
<code>poolparty</code>	<code>https://poolparty.<ref>.gw-pse.com/extractor/api/tag</code>	<code>https://poolparty.<sub>.gw-pse.com/extractor/api/tag</code>
<code>annotationGraph</code>	reference annotation graph URI	your annotation graph URI
<code>chunkAnnotationGraph</code>	reference chunk-annotation graph URI	your chunk annotation graph URI
<code>chunksGraph</code>	reference chunks graph URI	must match <code>chunksGraph</code> in Unstructured Ingest

Node 2 — Test: PoolParty Heartbeat (HTTP Request node) — URL is hardcoded, does NOT read from Config. Change to:

```
https://poolparty.<sub>.gw-pse.com/extractor/api/heartbeat
```

(Or, better, derive it from Config: `={{ $('Config').first().json.poolparty.replace('/tag', '/heartbeat') }}` — one place to change per deployment.)

Node 3 — Drop: Both Annotation Graphs (HTTP Request node) — SPARQL body is hardcoded. Replace both graph URIs to match your `annotationGraph` and `chunkAnnotationGraph` from Node 1:

```
DROP SILENT GRAPH <.../kg/annotations/> ;  
DROP SILENT GRAPH <.../kg/chunk-annotations/>
```

Node 4 — Query: All Text Nodes (HTTP Request node) — SPARQL body is hardcoded. Change the `chunkText` predicate to your ontology prefix:

```
?node <{your-ontology-prefix}chunkText> ?text .
```

Node 5 — Extract + Load: Chunks (Code node) — three hardcoded items:

1. **KP_BASE** constant — the base URI of concepts PoolParty returns. This comes from the PoolParty **project's own Base URI setting**, not the infrastructure subdomain. Get it from PoolParty → your project → Settings → Base URI:

```
const KP_BASE = 'https://.../'; // ← change this
```

2. **encodeURIComponent()** call inside **loadToGraphDB()** — ignores **cfg.chunkAnnotationGraph** and hardcodes the URI. Change to match your **chunkAnnotationGraph**.
3. **RDF triple predicate** — hardcoded in the triple-building string (**.../kg/ontology#hasConcept**). Change to match your ontology's concept relation predicate.

Node 6 — Verify: Annotation Counts (Code node) — graph URIs and prefix hardcoded. Update all three (**GRAPHS.structured** , **GRAPHS.chunks** , and the **PREFIX** declaration) to match your URIs. None of these read from the Config node.

Execute the workflow. One run tags both the structured nodes and the chunks, writing concept-relation triples into GraphDB. It drops and rebuilds both annotation graphs each run — safe to re-run when the taxonomy or extraction model changes.

Cross-workflow Consistency Requirements

Seven values must be consistent across multiple workflows. Set them once in one place, then copy to the others — inconsistency here causes silent failures (kNN misses, empty annotation graphs, 401s).

Value	Set in	Must match in
<code>esIndex</code> / <code>vectorIndex</code>	Unstructured Ingest Config	Configuration Code node
<code>embedModelId</code> / <code>embeddingsModelId</code>	Unstructured Ingest Config	Configuration Code node — kNN breaks if these differ even by whitespace
<code>chunksGraph</code>	Unstructured Ingest Config	Extractor Config
<code>annotationGraph</code>	Extractor Config	Extractor "Drop" node body, Extractor "Verify" GRAPHS object
<code>chunkAnnotationGraph</code>	Extractor Config	Extractor "Drop" node body, Extractor "Extract+Load" <code>encodeURIComponent()</code> call, Extractor "Verify" GRAPHS object
Ontology namespace URI	Structured Ingest prefixes + SPARQL Counts	Unstructured Ingest prefixes, Extractor "Query Text" SPARQL, Extractor "Extract+Load" triple predicate, Extractor "Verify" PREFIX
PoolParty concept base URI (<code>KP_BASE</code>)	Extractor "Extract+Load" Code node	PoolParty project Base URI setting — get the value FROM PoolParty, not the other way around

Steps 7–9 — Validate

Note: The notebooks referenced here are from a prior dataset and will not run against your data without modification. They illustrate the validation *approach* — how to use SPARQL and GraphDB's Workbench to confirm the ontology is loaded, triples are well-formed, Extractor annotations land in the correct graphs, embeddings are present, and kNN retrieval returns semantically grounded results. Adapt the SPARQL and assertions to your own ontology classes, graph URIs, and expected counts before running.

These steps save time. Catching a missing annotation graph, a mismatched embedding model, or an empty vector index here — at the GraphDB/SPARQL level — costs minutes. Discovering the same problem after activating the chatbot costs hours of tracing silent failures through the GraphRAG pipeline. Resolve data quality issues at this layer before proceeding to Step 10.

Step 7 — Validate the structured graph. Open `notebooks/Structured-Validate-GraphDB.ipynb` and run all cells. Confirms ontology, taxonomy, structured nodes, and Extractor annotations all loaded correctly into the expected named graphs.

Step 8 — Validate the unstructured graph. Open `notebooks/Unstructured-Validate-GraphDB.ipynb` and run all cells. Validates chunk nodes, embeddings, the Elasticsearch index field mapping, and GraphRAG readiness (should show **READY ✓** in the final cell).

Step 9 — Validate GraphRAG retrieval. Open `notebooks/GraphRAG-Prompt-Validation.ipynb` and run all cells. Puts real questions through the same two retrieval paths the chatbot uses — concept-layer SPARQL + vector kNN — and surfaces the grounding evidence you should expect to see cited in chatbot answers. This is the expected-grounding spec you compare the live chatbot against in Step 10.

Needs AWS Bedrock creds (environment or a local `.env`) to embed each question with Titan v2 for vector search. If missing, the vector stages skip and the concept-layer checks still run.

Step 10 — Activate the Chatbot

All 28 chatbot workflows were pre-seeded by the n8n database restore. With the graph and vector store now populated and validated, three steps bring the chatbot online:

1. Confirm all credentials in **Credentials** are filled (Step 3.6)
2. Confirm the **Configuration** node has the correct deployment values (Step 3.7)
3. **Activate** the `Main` workflow (toggle the activation switch)

Smoke test: ask a representative question for your domain — it should return a grounded, cited answer in around 90 seconds or less. Compare its concepts and cited sources against the expected grounding from `GraphRAG-Prompt-Validation.ipynb` (Step 9).

- **Hangs forever:** workflows not seeded, or an LLM credential is empty
- **Returns no sources:** `vectorStorePreset` is wrong, or `vectorIndex` doesn't match the actual Elasticsearch index

LLM model IDs must be Bedrock inference profile IDs (e.g. `us.anthropic.claude-sonnet-4-6`) — bare foundation-model IDs return `InvalidRequestException`. All three LLM slots in `Configuration` should use the same profile ID, and it must be one your account/region has enabled (needs IAM `bedrock:InvokeModel*` on the profile and the per-region foundation-model ARNs, plus `aws-marketplace:Subscribe` to enable model access).

Step 11 — Prune n8n Execution History

Execution records accumulate quickly during active ingest runs and slow the n8n UI. The **N8N Prune Database** workflow deletes them. It may already be present in the seeded database; if not, **Import from File**

`n8n-workflows/cleanup.json` .

The `n8n Internals` credential was created in Step 3.5. Assign it to both the **Get many executions** and **Delete an execution** nodes in this workflow.

Why the `/api/v1` suffix matters: the `n8n-nodes-base.n8n` node type appends only the bare resource path (`/executions`) to the credential's Base URL — it does not add `/api/v1` itself. A Base URL of `http://graphrag-workflows:5678` (without `/api/v1`) makes the node call `/executions` and return **404** ("The resource you are requesting could not be found") even though the API is up and the key is valid. That is why `n8n Internals` uses `http://graphrag-workflows:5678/api/v1` .

In the **Get many executions** node, enable **Return All** — without it only the first ~100 records are fetched.

In **Set Executions to Keep**, set `executionsToKeep` to `0` to purge everything, or a positive integer to keep that many recent records per workflow. Then execute the workflow.

The delete node returns each deleted record with `deletedAt: null` — expected for a hard delete (the field is null before removal; the record is gone from the DB). If the execution count was very large, run the workflow a second time to catch records beyond the first full page.

Re-run Reference

What changed	Action
CSV / structured source data	Re-run Structured Ingest, then Extractor
Taxonomy or extraction model	Rebuild Extraction Model in Graph Modeling, then re-run Extractor
Source documents	Re-run Unstructured Ingest → Extractor
<code>helm upgrade</code> / <code>reset-helm.sh</code> ran	Re-run Steps 3.3 and 3.9 (env injection + PVC mount were wiped)
Stack rebuilt (destroy → apply)	Full Chapter 2 from Step 3.2 (Secret gone, env gone, API key stale)

POC Troubleshooting

Symptom	Likely cause	Fix
Workflow can't read <code>\$env.*</code>	env not injected or wiped by helm upgrade	Re-run Step 3.3 + <code>kubectl -n graphrag rollout restart deploy/graphrag-workflows</code>
Internal API calls return 401	API key not rotated or not updated in Data Table	Redo Step 3.5
Graph Modeling heartbeat 401	<code>EXTRACTOR_AUTH</code> was <code>echo</code> - encoded (trailing newline)	Re-encode with <code>printf</code> , re-create the Secret (Step 3.2), re-inject env (Step 3.3)
Extractor: <code>HTTP 400: Concept Index is empty</code>	Extraction model not rebuilt after thesaurus import	Graph Modeling → Corpus → Rebuild Extraction Model
Graph Modeling returns docs with no concepts	extraction model not rebuilt	Corpus → Rebuild Extraction Model
GraphDB load 400 "Illegal carriage return"	raw newline in a Turtle literal	fixed in current workflows; re-import the latest JSON
Unstructured Ingest: ENOENT on staging files	Staging PVC not mounted (wiped by helm upgrade)	Re-run Step 3.9
Notebook: "Cannot reach Elasticsearch"	ES unreachable	confirm <code>esUrl</code> in Unstructured Ingest Config points to the in-cluster service
N8N Prune → 404 "resource not found"	<code>n8n Internals</code> credential missing <code>/api/v1</code> in Base URL	Set Base URL to <code>http://graphrag-workflows:5678/api/v1</code> (Step 11)
Prune workflow runs but nothing deleted	<code>Return All</code> not enabled — only the first page fetched	Enable Return All on the <code>Get many executions</code> node (Step 11)
Chatbot spins forever	Workflows not seeded, or LLM credential empty	Confirm restore ran; fill Main LLM Model credentials (Step 3.6)
Chatbot answers with no sources	<code>vectorStorePreset</code> wrong or <code>vectorIndex</code> mismatch	Set <code>elasticsearch_native</code> + correct index name in Configuration node (Step 3.7)